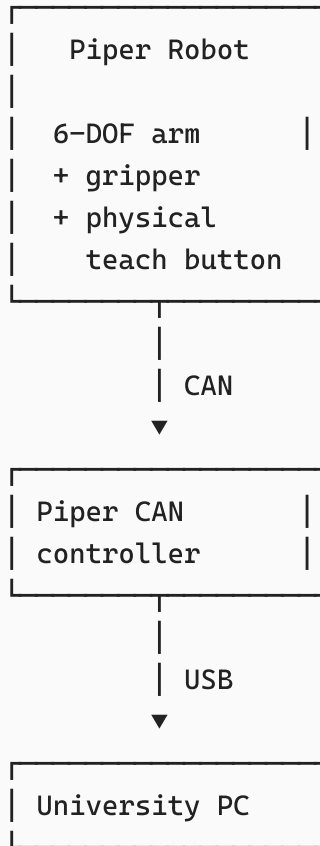


1. What you are actually trying to achieve

Your physical Piper setup is roughly:



Your university PC currently communicates with the Piper through the CAN interface.

You want to replace:

```
Physical button  
↓  
double tap  
↓  
Piper replays recorded trajectory
```

with:

```
Laptop  
↓  
Python program / SDK  
↓  
CAN  
↓  
Piper
```



replay recorded trajectory

In other words:

You want software to trigger the same functional operation that you currently initiate through the physical double-tap.

That is the core project.

2. What the physical button is actually doing

This distinction is extremely important.

The button isn't magically "moving the robot."

There are two conceptual operations involved.

Teaching / recording

You manually hold the robot arm and move it through some trajectory.

For example:

Position A



Position B



Position C



Position D

The Piper records that trajectory.

Then you finish the teaching operation.

The robot now has a recorded trajectory.

Playback

When you perform the appropriate physical button operation — in your case, the double tap — the robot enters the trajectory reproduction operation:

Recorded trajectory



```
Piper
  ↓
A → B → C → D
```

So the physical button is essentially providing a **human interface to a firmware function**.

That is why your instinct was:

"Can I somehow trigger that button through CAN?"

That is a reasonable question.

But we don't necessarily want to reproduce the **electrical button press**.

3. The first major correction to your approach

Your original thought was approximately:

```
Laptop
  ↓
CAN
  ↓
somehow simulate double-click
  ↓
physical button function
```

We should instead aim for:

```
Laptop
  ↓
CAN
  ↓
Piper command
  ↓
"Execute taught trajectory"
```

Why?

Because AgileX's SDK already exposes a command specifically for this.

The current `piper_sdk` source contains:

```
MotionCtrl_1()
```

which uses CAN ID:

0x150

and has a field called:

```
grag_teach_ctrl
```

for drag-teaching control.

The documented values include:

```
0x00 → no operation
0x01 → start teaching recording
0x02 → finish teaching recording
0x03 → execute taught trajectory
0x04 → pause execution
0x05 → continue execution
0x06 → terminate execution
0x07 → move to trajectory start point
```

([GitHub](#))

So there is already a software-level representation of the operation we're interested in.

Conceptually:

```
Physical interaction
    ↓
Firmware
    ↓
"Execute taught trajectory"

Software interaction
    ↓
CAN command
    ↓
"Execute taught trajectory"
```

That's much cleaner.

4. But there is an important subtlety

This is where I want you to be careful.

There are **two different questions**:

Question A

Can the robot execute a previously recorded teaching trajectory through CAN/software?

Yes, the SDK exposes exactly such a command.

Question B

Can software completely replace every aspect of the physical button's teaching-mode behavior?

Don't assume that yet.

There is an AgileX GitHub issue specifically discussing this.

AgileX stated that, with the relevant firmware behavior, the **button-triggered teaching mode has priority**, and that entering the button teaching mode itself was not exposed through the protocol in that context. ([GitHub](#))

This matters because we don't want to conflate:

```
START recording
```

with:

```
PLAY recorded trajectory
```

The SDK has commands for teaching control, but the exact behavior of your **specific Piper firmware + physical button + firmware version** must be verified.

Therefore, later, when we have the robot, we'll experimentally establish:

```
physical button
  ↓
what CAN traffic occurs?
  ↓
what firmware state changes?
  ↓
what SDK command reproduces it?
```

That's much safer than assuming.

5. Your CAN idea is still fundamentally correct

Your mentor telling you:

"Through the Python SDK you can have total control over the robot"

is directionally correct.

The Piper SDK is designed to communicate with the arm through CAN and exposes functionality for things such as:

- reading joint states
- reading motor/driver states
- controlling joints
- controlling end pose
- controlling the gripper
- enabling/disabling
- trajectory control
- teaching trajectory control
- emergency-stop related functions
- reading robot status

AgileX provides official demos covering things such as joint control, end-pose control, gripper control, enable/disable and other operations. ([GitHub](#))

So we're not trying to reverse engineer the entire robot from scratch.

We're trying to learn how to interface with an **already documented robot API**.

That's a much better project.

6. The CAN controller is the bridge

Think of CAN as the communication highway.

Your Python program doesn't directly manipulate the robot motors.

Instead:

```
Python
  ↓
Piper SDK
  ↓
python-can
  ↓
CAN interface
  ↓
USB-CAN hardware
  ↓
CAN bus
```

↓
Piper controller
↓
Robot firmware
↓
Motors

So when we eventually execute something like:

```
execute taught trajectory
```

the chain is approximately:

Python instruction
↓
SDK converts it to CAN message
↓
USB-CAN sends CAN frame
↓
Piper receives frame
↓
Piper firmware interprets frame
↓
Piper executes command

This is why knowing the CAN layer matters.

7. There are actually TWO software layers we need to understand

This is another important distinction.

Layer 1 — High-level SDK

Something like:

```
robot.some_function(...)
```

You don't need to manually construct every CAN byte.

The SDK handles that.

Layer 2 — CAN protocol

Underneath, the SDK eventually generates CAN messages.

For example, AgileX documents:

```
CAN ID: 0x150
```

for `MotionCtrl_1`. ([GitHub](#))

The SDK internally creates the appropriate CAN message and sends it.

So our development strategy is:

First:

```
Python
↓
SDK
↓
CAN
```

rather than immediately:

```
Python
↓
manually constructed hexadecimal CAN frames
```

We only go lower-level when we actually need to understand/debug something.

8. Why we want to observe CAN traffic later

When you return to the robot, we're going to have a very useful experiment available.

Suppose:

```
Robot is ready
Recorded trajectory exists
```

We start a passive CAN observation.

Then:

```
Person double taps button
```

We observe:

```
CAN frame
CAN frame
CAN frame
CAN frame
...
```

Then we repeat.

We can compare the traffic.

The objective is to determine:

```
button action
  ↓
CAN communication
  ↓
specific command
  ↓
robot response
```

If we see the relevant command correspond to the documented teaching-trajectory execution function, we've confirmed the mapping.

That gives us **empirical evidence** instead of guessing.

9. Why we're not going to immediately send CAN commands

This is critical.

CAN gives us a direct path into the robot's control system.

A bad command isn't like sending the wrong parameter to a harmless desktop program.

A robot can physically move.

The Piper SDK itself explicitly warns that advanced control protocols can damage the arm if misused. ([GitHub](#))

There is an especially nasty example in the SDK:

```
ResetPiper()
```

AgileX documents that resetting Piper can immediately remove motor power and cause the arm to fall. ([GitHub](#))

So our philosophy is:

Observe → understand → test safely → control.

Not:

Install SDK → randomly call functions → see what happens.

10. The complete project strategy

This is the roadmap we've established.

PHASE 0 — Understand the architecture

We are here right now.

Understand:

```
Physical Piper
  ↓
CAN
  ↓
USB interface
  ↓
Laptop
  ↓
Python SDK
```

And understand the difference between:

```
teaching
```

and:

```
trajectory playback
```

PHASE 1 — Prepare the laptop

No robot.

No CAN.

No motor movement.

We're just preparing the development environment.

Your current machine reports:

```
Python 3.13.5
pip 26.2.1
Git 2.54.0
```

So:

Python

```
3.13.5
```

pip

```
26.2.1
```

Git

```
2.54.0
```

That's a perfectly usable development environment.

And importantly, the **current** `pyAgxArm` **documentation explicitly supports Windows 10/11 and Python 3.6+ through 3.14**, so Python 3.13.5 fits within its stated compatibility range. ([GitHub](#))

11. Why we're choosing Windows

Earlier I said Ubuntu would be more natural for robotics.

That's still true generally.

Linux has a beautiful robotics ecosystem:

```
SocketCAN
ROS
ROS 2
can-utils
Linux networking tools
```

But that does **not** mean you need to abandon Windows for this project.

The current AgileX `pyAgxArm` documentation explicitly provides:

```
Windows 10/11
```

support. ([GitHub](#))

For Windows, AgileX specifies:

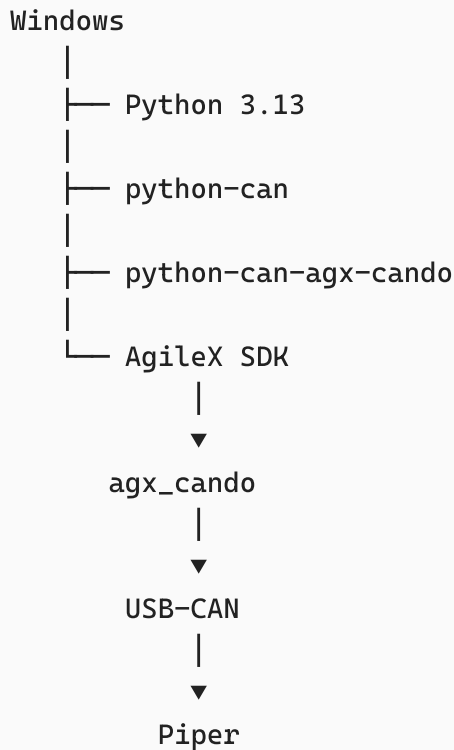
```
interface = "agx_cando"  
channel = "0"
```

and requires the:

```
python-can-agx-cando
```

plugin. ([GitHub](#))

So our Windows architecture will eventually be:



The `python-can-agx-cando` project is specifically a Windows backend that communicates with AgileX's `cando.dll`. It also automatically selects the appropriate 32/64-bit DLL based on the Python process architecture and documents Python 3.6+ support. ([GitHub](#))

So **Windows is not some hacky workaround anymore.**

12. There is another SDK distinction

This is important because you will encounter two names.

Older/current `piper_sdk`

Repository:

```
agilexrobotics/piper_sdk
```

This is the dedicated Piper SDK you've probably heard about.

It contains functions such as:

```
MotionCtrl_1()  
MotionCtrl_2()  
...
```

and directly exposes the teaching trajectory control we discussed. ([GitHub](#))

Newer `pyAgxArm`

AgileX now also has:

```
pyAgxArm
```

which is a more general Python SDK for AgileX arms and end effectors.

It explicitly supports:

```
Piper  
Nero  
AgxGripper  
Revo2
```

and Windows/Linux/macOS. ([GitHub](#))

For Piper specifically, AgileX has current Piper demos in this newer SDK as well. For example, the repository contains a Piper demo that selects `PIPER_H` and configures Windows with:

```
agx_cando  
channel "0"
```

([GitHub](#))

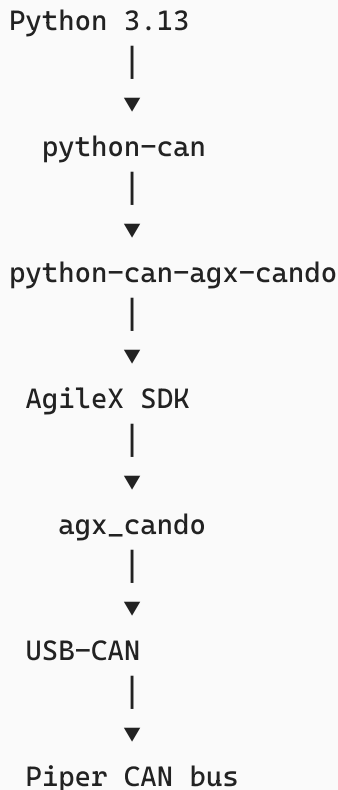
So we shouldn't blindly install every package we find.

We'll first establish **which SDK version/framework matches the firmware and setup in your university's Piper**.

That's why we're deliberately stopping at the prerequisite stage.

13. The eventual Windows software stack

Conceptually, we'll have:



We will verify each layer independently.

14. What we will NOT do

This is just as important as what we will do.

We will **not** initially:

- send arbitrary CAN frames
- experiment with motor commands
- use MIT joint control
- change firmware
- modify robot parameters

- change motor limits
- reset the robot
- disable safety mechanisms
- try random CAN IDs
- emulate the physical button electrically
- start trajectory playback without understanding the state

The goal is not:

"Let's see if this command works."

The goal is:

"Let's know exactly what command we are sending, what subsystem receives it, and what physical consequence it has before transmitting it."

15. The eventual testing ladder

When we finally connect the robot, we'll go from least dangerous to most consequential.

Level 1 — Detect hardware

```
Laptop
  ↓
USB-CAN
```

Verify Windows sees the CAN hardware.

No robot movement.

Level 2 — Establish CAN communication

```
Laptop
  ↓
CAN interface
  ↓
Piper
```

Confirm the CAN interface initializes correctly.

Level 3 — Receive-only observation

Read:

```
CAN frames
```

without sending motion commands.

We'll determine what the robot is broadcasting.

Level 4 — Read robot state

Use SDK functions to obtain things like:

```
joint positions  
motor state  
robot status
```

No deliberate motion.

Level 5 — Verify control state

Determine whether the robot is:

```
enabled  
disabled  
CAN-controlled  
teach mode  
idle  
executing trajectory
```

and understand what each state means.

Level 6 — Test the teaching playback command

Only after the above is understood:

```
existing recorded trajectory  
  ↓  
software trigger
```

↓
Piper playback

This is the actual target.

16. The final intended system

Eventually, your laptop could have a Python program like:

```
Piper Controller
```

```
[Connect]
```

```
Robot status:
```

```
    Connected
```

```
    Enabled
```

```
    Idle
```

```
Recorded trajectory:
```

```
    Available
```

```
[ Execute Recorded Trajectory ]
```

```
[ Pause ]
```

```
[ Continue ]
```

```
[ Stop ]
```

Underneath:

```
Button press
```

would no longer be the only way to initiate the playback.

Your program could perform the corresponding SDK operation.

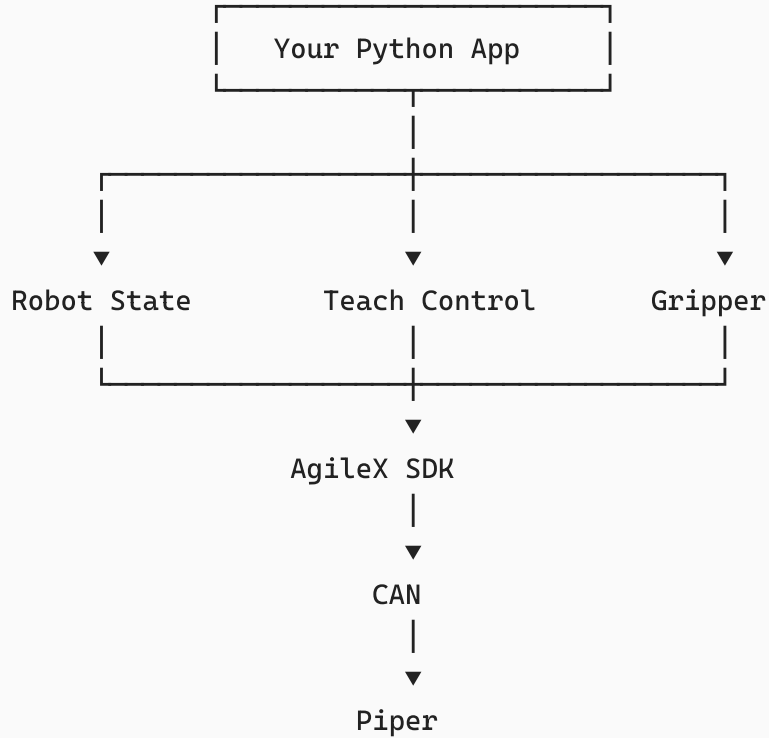
That is the actual engineering outcome we're aiming for.

17. And there is a potentially bigger project hiding here

Once we establish reliable SDK control, your project isn't limited to:

"Replace double-click with laptop."

You could potentially build a software control layer:



Then you can eventually have:

```
Record trajectory
  ↓
Save trajectory metadata
  ↓
Select trajectory
  ↓
Execute
  ↓
Pause
  ↓
Resume
  ↓
Stop
```

That becomes an actual **robot control application**, rather than just a button emulator.

But that's later.

18. Where we currently stand

We have completed the **conceptual planning**.

Your original idea

Can I control the physical double-tap through CAN?

Refined engineering question

Can I use the Piper's CAN interface and AgileX SDK to trigger the firmware operation corresponding to execution/reproduction of an existing drag-teaching trajectory?

Yes, that is the right question.

And the SDK provides a documented `grag_teach_ctrl = 0x03` operation for executing the taught trajectory. ([GitHub](#))

But we will **verify your specific firmware/button behavior experimentally** rather than assuming the physical double-click maps one-to-one to that command.

19. Your current laptop status

You have:

OS:
Windows

Python:
3.13.5

pip:
26.2.1

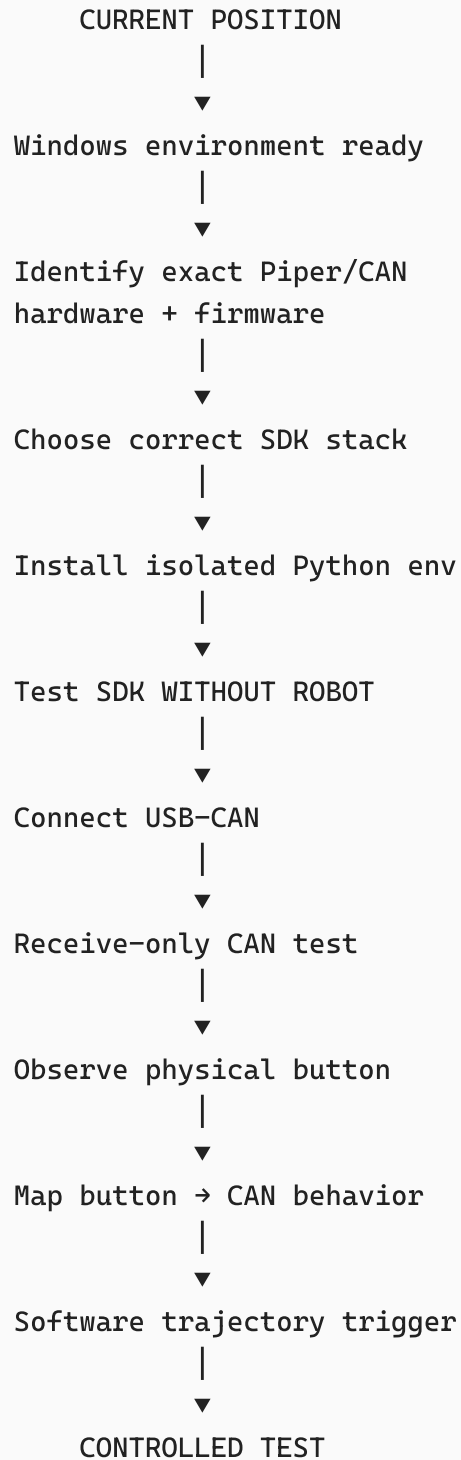
Git:
2.54.0

So you're already at the beginning of the software-preparation stage.

And one thing I particularly **do not** want you to do yet is start installing random versions of `piper_sdk`, `pyAgxArm`, `python-can`, or CAN drivers.

We now know there are **multiple AgileX SDK generations and Windows-specific CAN backends**, so we'll choose the correct stack deliberately. The current `pyAgxArm` documentation is explicit that Windows uses `agx_cando`, while the older dedicated `piper_sdk` repository has historically documented Linux-oriented CAN setup. ([GitHub](#))

So the next actual implementation phase is:



That's the strategy.

And importantly, **we haven't touched the robot yet, so we've taken essentially zero robot-side risk.** The next time we proceed, we'll do it one step at a time rather than dumping twenty commands on you.